



UNIVERSITY
OF HULL

Live Software Updates in a Real-Time System

Nathan Rignall

This dissertation is submitted on April, 2024

Word count: 1976

Abstract

My abstract ...

Acknowledgements

My acknowledgements ...

Contents

1	Introduction	8
1.1	Project Background	8
1.2	Objectives	8
1.3	Research Question	8
2	Background	9
2.1	Real-Time	9
2.2	Control Systems	9
2.3	Operating Systems	9
2.4	Mixed Criticality	10
2.5	Middleware	10
2.6	Software Updates	10
3	Literature Review	11
3.1	Review	11
3.2	Case Study	11
3.2.1	Control Systems	11
3.2.2	Dynamic Updates	11
3.2.3	Disruption-free	11
3.2.4	A model	11
3.2.5	Grid-connected	12
3.3	Rational	12
4	Requirements	13
5	Design	15
5.1	Operation	15
6	Runtime	16
7	Implementation	19

7.1	Hardware	19
7.1.1	Test Harness	20
7.1.2	Development Cards	23
8	Testing	26
9	Evaluation	27
10	Conclusion	28
	References	29
A	Extra Information	30

Chapter 1

Introduction

1.1 Project Background

1.2 Objectives

This project seeks to develop multiple methods and system architectures for live software updates in a real-time system via an example application. Using a timing analysis solution, such as a logic analyser, this will be tested to ensure that the real-time application meets all imposed deadlines (defined latency and jitter). The goal of this is to prove that it is feasibly possible to update software on a live system whilst maintaining real-time guarantees. Although this is a complex task the intention is to achieve this before February.

Another primary objective is to implement a reliable external testing method for a real-time environment, using existing research. This will allow for accurate measurement of the real-time behaviour of the program at all stages of the update, to validate that the upgrade is successful. Minimal external hardware is required, and the plan is to achieve this before February.

1.3 Research Question

Chapter 2

Background

2.1 Real-Time

A real-time system is defined as a system which requires correct logical operation with completion in a configured deadline. This is in contrast to a non-real-time system which correct operation is desired but lacks configured deadlines. Kavi, Akl, and Hurson 2009

These real-time systems can be categorised into hard and soft. Hard real-time systems contain strict timing constraints, where a response must be calculated in a specified time-frame and a missed deadline could result in a catastrophic outcome. Soft real-time systems also have timing constraints, but a missed deadline should not result in a catastrophic outcome.

Typical hard real-time systems include aircraft or robotic control computers and soft real-time systems include media players or gaming computers. Soft real-time systems can be used in control applications, but the soft timing must be considered in the system design.

2.2 Control Systems

A control system Differential equation

2.3 Operating Systems

An Operating System (OS) is a piece of software that controls the execution of application programs on a system. Usually this involves managing resource allocations, such as a

processor execution time, memory usage and generic inputs/outputs. Stallings 2018

In order to develop a real-time applications with an OS, a RTOS (Real-Time Operating System) is usually required. This is to ensure the time deadlines of an application are enforced.

Hard real time operating system are designed to run applications with absolute timing Worst-Case Execution Time Assigned a relative deadline, all tasks must complete in their deadline. Modelled mathematically to ensure that all tasks run to completion

Define the maximum delay after a deadline

Scheduling Memory

UNIX, POSIX

2.4 Mixed Criticality

2.5 Middleware

2.6 Software Updates

Chapter 3

Literature Review

3.1 Review

Analysis of existing dynamic software updating techniques for safe and secure industrial control systems

Challenges in Dynamic Software Updating

A survey of dynamic software updating

3.2 Case Study

3.2.1 Control Systems

Live Seamless Firmware Upgrade in Real Time Control Systems

3.2.2 Dynamic Updates

Dynamic Software Updates for Real-Time Systems.

3.2.3 Disruption-free

Disruption-free software updates in automation systems

3.2.4 A model

A Model for Updating Real-Time Applications

3.2.5 Grid-connected

Enhanced Uptime and Firmware Cybersecurity for Grid-Connected Power Electronics

3.3 Rational

Chapter 4

Requirements

The project's primary objective is to develop multiple system architectures to enable live software updates in a running control real-time system.

Framework for developers to run software components which can be dynamically updated

Example application that uses one differential equation and requires its state to be maintained with a plant for the example application to test its functionality

Testing harness that can measure the

High-Level During operation

The software infrastructure must provide the functionality to run software components with a cyclic schedule allow users to create/start/stop/remove software components during operation allow users to change the configured cyclic schedule during operation

provide the functionality to allow components to communicate using messages passing allow users to configure communications between components using routes and channels allow users to add/remove routes between software components during operation

Low-Level create - loads the software component binary - starts the software component

start - marks the software component as ready to run

stop - marks the software components as not ready to run

stop

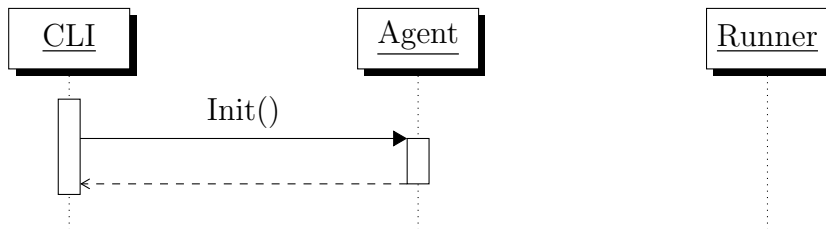
The example application must maintained

The plant must model a mass falling due to gravity

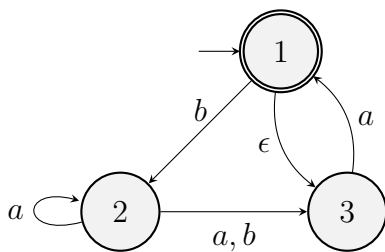
Chapter 5

Design

5.1 Operation



ClassName
name : attribute type
name : attribute type = default value
name(parameter list) : type of value returned
<i>name(parameters list) : type of value returned</i>



What is Linux Unix process model How containers work What is Xen

Chapter 6

Runtime

To select the most optimal environment to develop the software infrastructure upon, a series of benchmarking activities were carried out. These focused on evaluating operating systems and hypervisors which could operate in a real time environment. The search was restricted to free and open source software which supports the Raspberry Pi Compute Module 4.

EXPLAIN THE VIRTUALISATION EXTENSIONS AND VIRTUALISATION

(Abeni and Faggioli 2020) compiled a similar study into the real time performance of open source hypervisors. Comparing KVM and Xen on various intel x86 processors.

SUMMARISE THE WORK HERE

For the purpose of this project the real-time performance on an Arm64 target, the chosen development target, must be specifically analysed. It is reasonable to expect that the performance characteristics may be different between the processor architectures due to the implementation of virtualisation extensions. This analysis shall compare the behaviour of 'bare metal' Linux and the Xen hypervisor, not including KVM.

This is because KVM is implemented ontop of the Linux kernel.

To reap the benefits of dynamic updates it is preferred that the hypervisor is independent of the operating system (explain why).

Also, in a true real-time system it is unusual to use linux. Rather a more preferable OS such as Zephyr or FreeRTOS (Free and open source solutions used).

Comparing straight linux and linux + Xen should give us a good method to estimate the specific latencies brought about by Xen.

EXPLAIN RT LINUX

Explain Yocto and Bitbake here too?

The RT-Tests suite includes a cyclicttest program. This is a simple yet effective program that can be used to estimate the real time latencies in a system. It measures real time jitter by creating a interrupt event at a specific time in the future and measuring the time delay between the requested wake-up and actual wake-up time. While this provides a good method for identifying operating system induced latency and jitter it does not isolate latencies induced by hardware or low-level firmware. At this stage it can be assumed that the selected development targets do not induce significant latencies.

Under load rt-characteristics change

To set a baseline we shall first compare the real time behaviour of Linux with the standard Preemnt and Preempt RT kernels.

There is little difference between the two kernels when the system is mostly idle. However when the system is under load the differences between the kernel's schedulers becomes apparent. Especially so for memory stress versus cpu stress.

Now we are satisfied that linux, with the real time kernel patch performs adequately we can begin configuring Xen.

Exceptionally poor performance at first. Assume that is due to the nature of the Xen scheduler implementation.

EXPLAIN THE XEN SCHEDULER OPTIONS

When using the None scheduler these latencies and jitter are significantly improved

Multi core interference

The first experiments were performed on the linux kernel running without a hypervisor.

The next experiments used a real-time linux kernel.

Xen Scheduler (Credit vs)

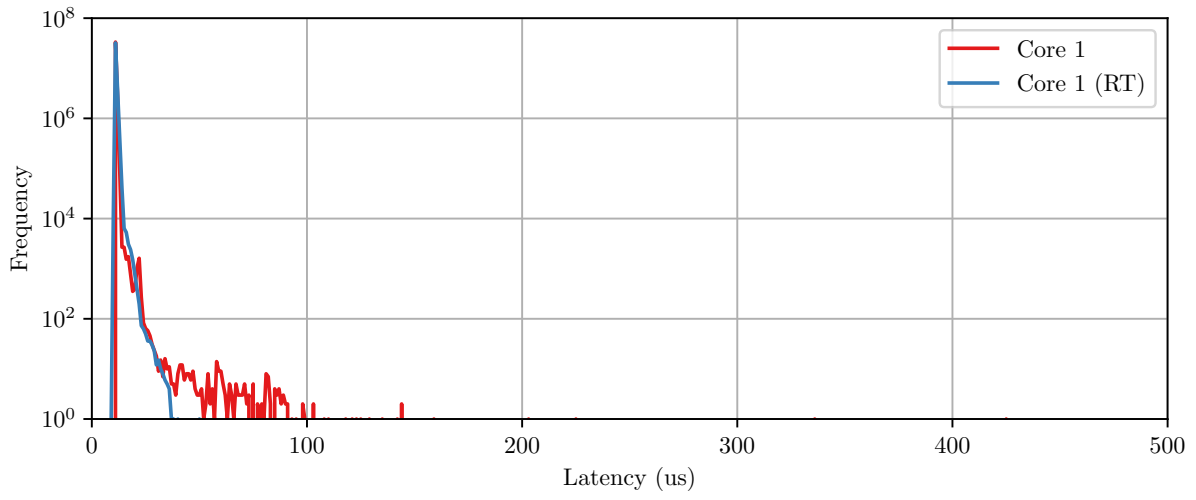


Figure 6.1: Latency Graph

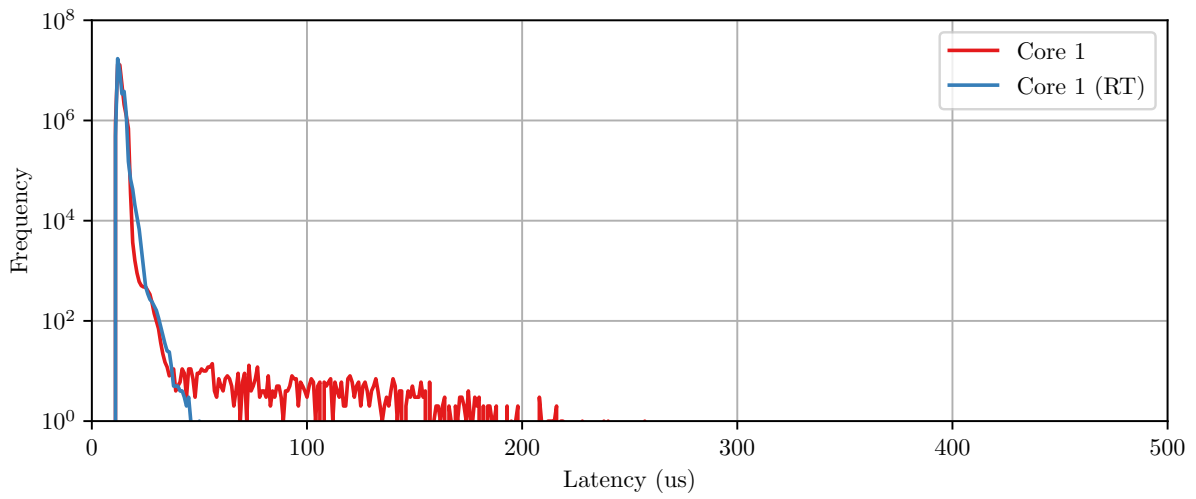


Figure 6.2: Latency Graph

Chapter 7

Implementation

Scheduler

To implement an event based scheduler it must be able to send signals between processes without significant jitter and latency. The level of jitter that is acceptable can be estimated by calculating a worst case execution time for a control loop in a particular application.

There are several different methods that can be used to send signals between processes in a Linux system. Unix shared memory and sockets shall both be investigated as they can be easily transported onto a non-linux POSIX system.

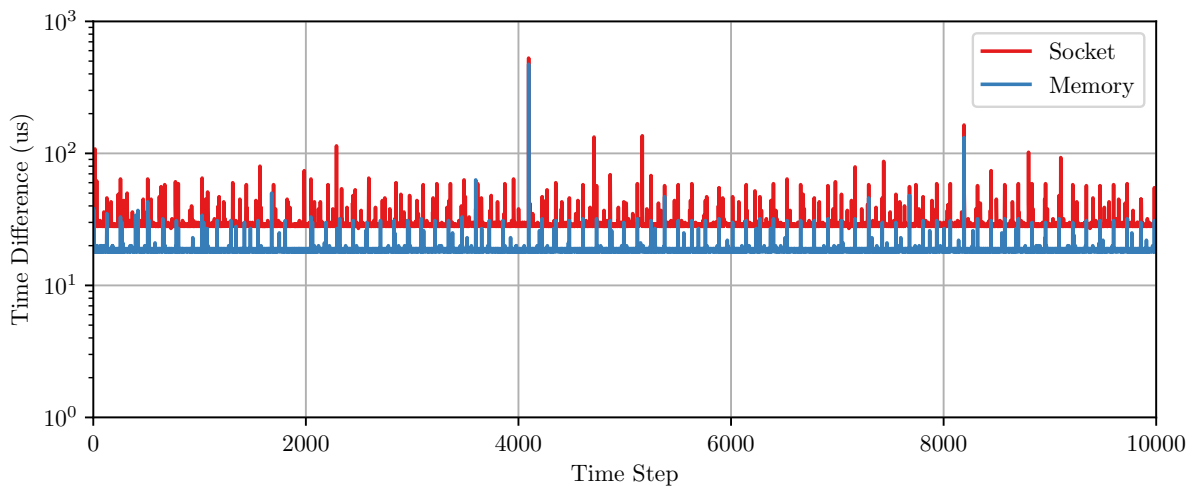


Figure 7.1: Compare Execution Latency

7.1 Hardware

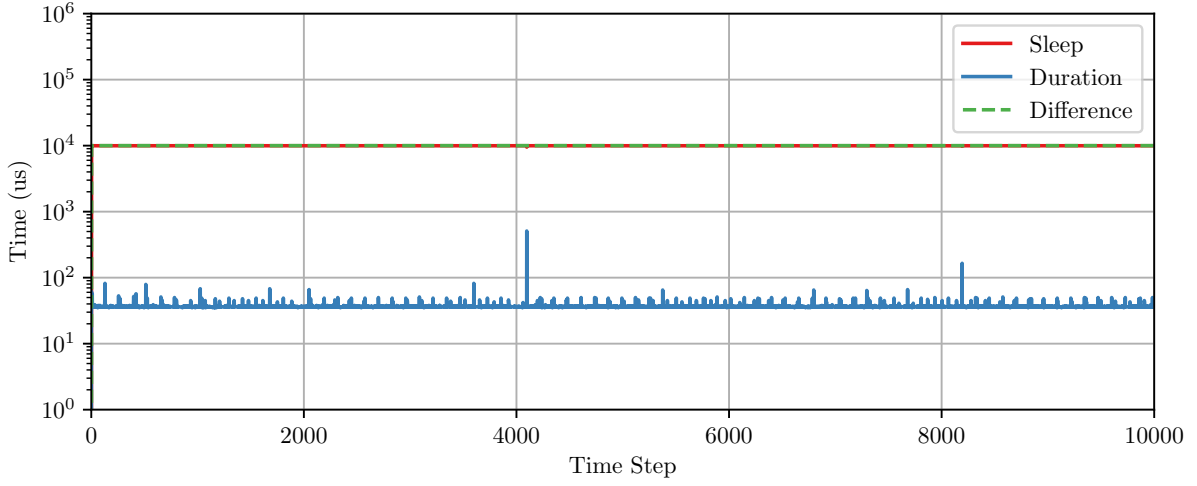


Figure 7.2: Shared Memory Loop Execution

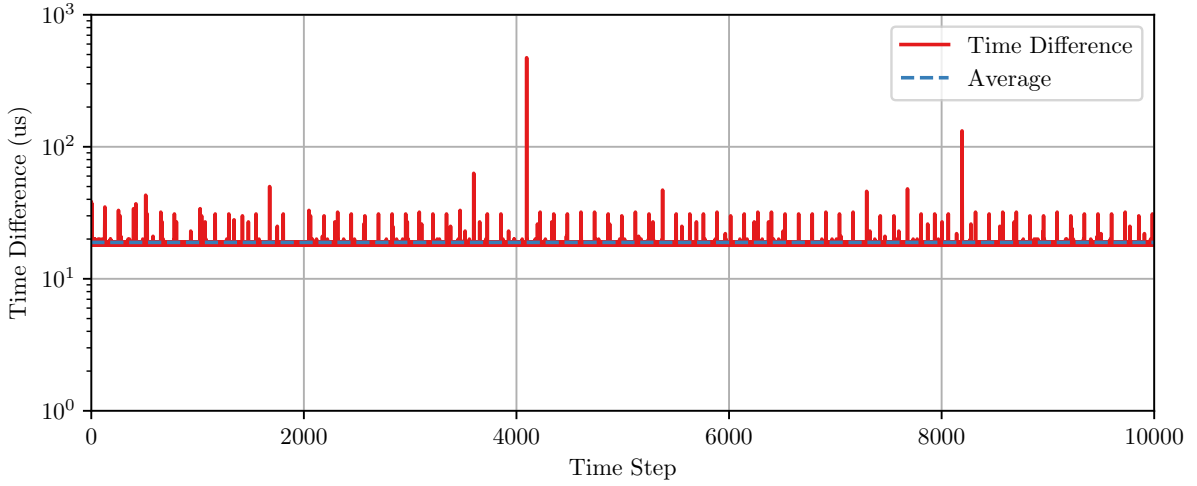


Figure 7.3: Shared Memory Execution Latency

7.1.1 Test Harness

To accurately test the real time behaviour of the system, an external test harness is required. While latencies and jitter can be measured on-target with instrumentation, this does not independently measure the system's performance. Though this harness, an example control loop implementation can be simulated to ensure all deadlines are met during a live software update.

Figure 7.7 contains a signal generator, this randomly produces a signal and outputs a packet. These packets are ingested by the software infrastructure where processing occurs. An output packet is subsequently generated (mirroring the input) and sent to a signal receiver, which captures this packet.

The signal generator and receiver output their current state over discrete digital signals

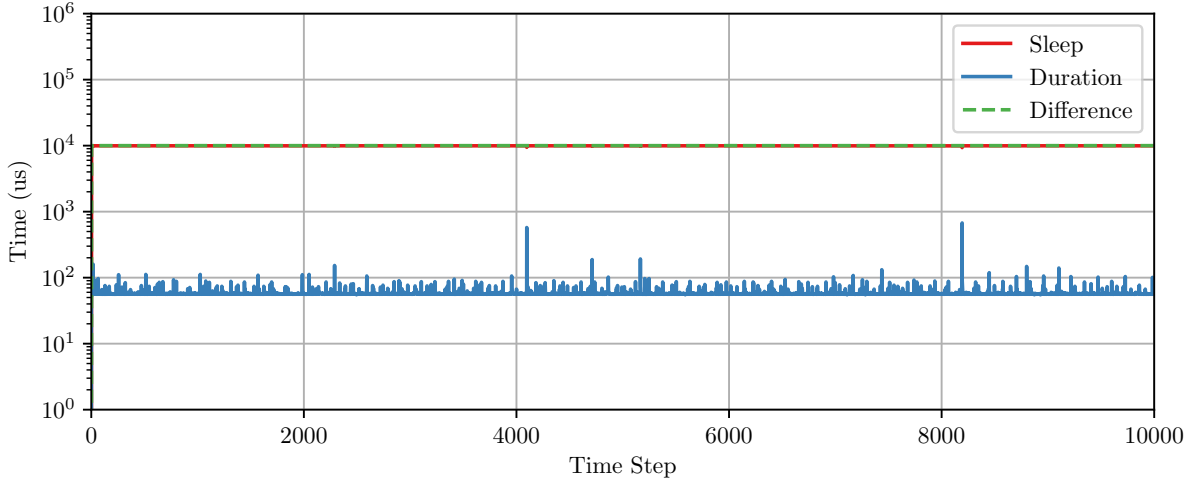


Figure 7.4: Socket Loop Execution

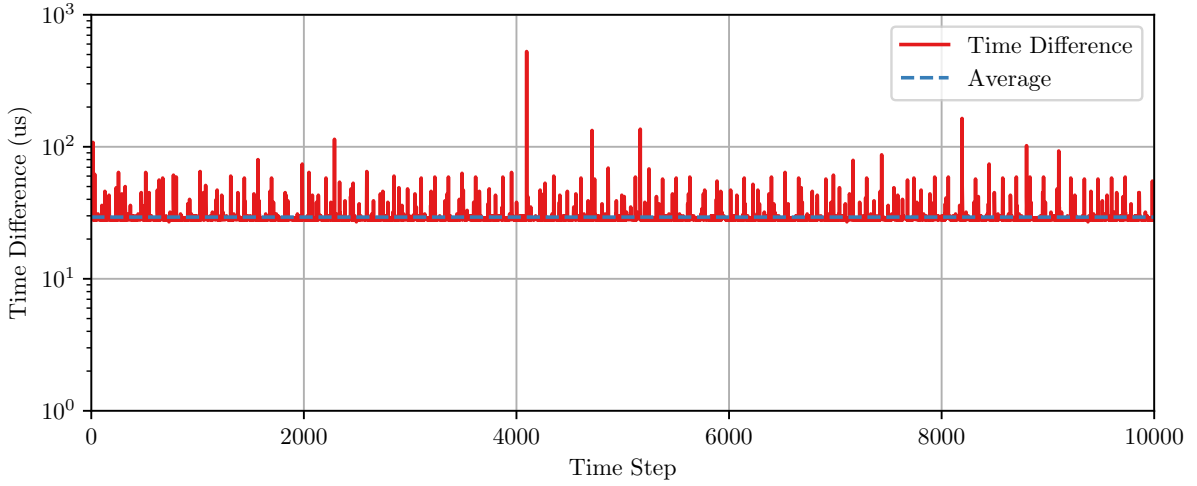


Figure 7.5: Socket Execution Latency

so that a logic analyser can record the state of the system over time. Specifically the states can be compared to measure the complete system latency during standard operation and software update. If the generator and receiver produce additional reference clock signals, the logic analyser can more closely calculate the software infrastructure induced latency.

The generator, receiver and logic analyser were all implemented using the RP2040 micro-processor from Raspberry Pi for hardware simplification.

The two controllers that performed packet communication used the W5500 chip to send UDP packets over ethernet, this integrates nicely with Elafry messages. This co-processor offloads the complex ethernet drivers and communicates to the main processor over

This communicates over SPI and offload sthe compelx ethernet stuff onto this co-processor.

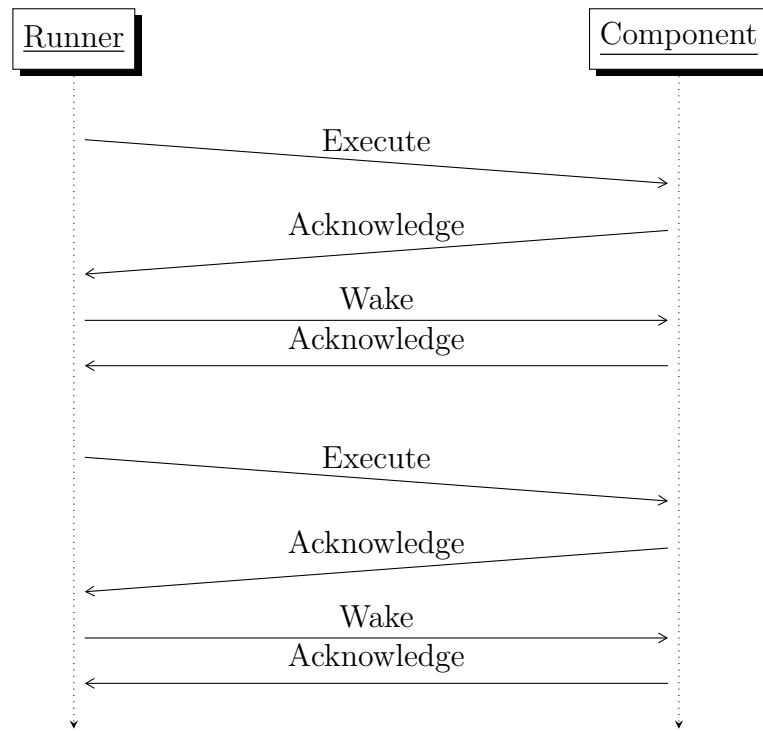


Figure 7.6: Client-Server messaging

SPI signals configure the IP address of the target device and allow the user to send data.

The intention for this software was to also use Rust, however we could not get the Rust libraries for the W5500 working quickly, sure with extra time with could have been possible. To simplify this software the arduino development environment was used. This as all the drivers for the w5500 are integrated in arduino libraries.

Logic analyser <https://github.com/dotcypress/ula>

Capture the status of the 16 input channels on the Pi Pico at specified frequencies. When integrated with the Pulseview analysis software we can identify the latencies

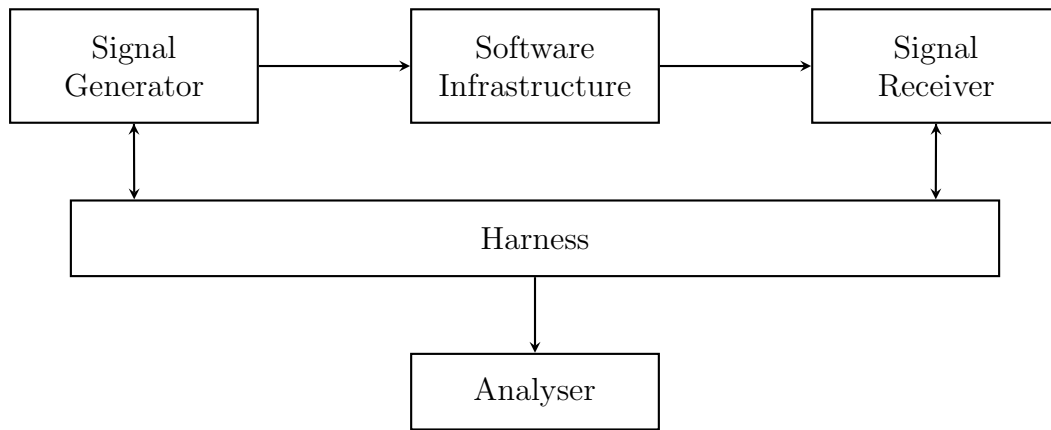


Figure 7.7: Test Harness Design

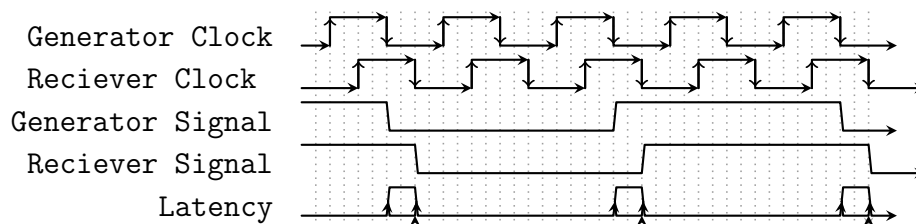


Figure 7.8: Test Harness Capture

7.1.2 Development Cards

In order to build and test the real time software, dedicated development hardware is required to ensure an isolated testing environment.

These test cards consist of a Raspberry Pi Compute Module 4 (CM4). This contains the same overall hardware as the standard raspberry pi 4. EMMC flash for high vibration environments. Exception being the ethernet IC and EMMC flash (This later caused problems with compiling the operating system and bootloader)

Key requirement is remote debugging, full remote power and data control. Also placed in a rack Selected the Pi KVM project for remote control as it supports deploy as well as additional GPIO control.

The CM4's display is captured using the CSI port on the KVM with a HDMI to CSI converter. The power is controlled using a relay, that controls the 12v supply to the CM4. The signal flags of the EMMC storage are directly connected to the KVM. EXPLAIN the signal flags These are configured in the KVM user interface UART is connected to a USB serial converter. USB host is connected to the KVM for flashing the CM4. Finally a debugger (JTAG) in case of modifications to the kernel itself.

Two of these so that multiple benchmarking activities can be performed at once and compared.

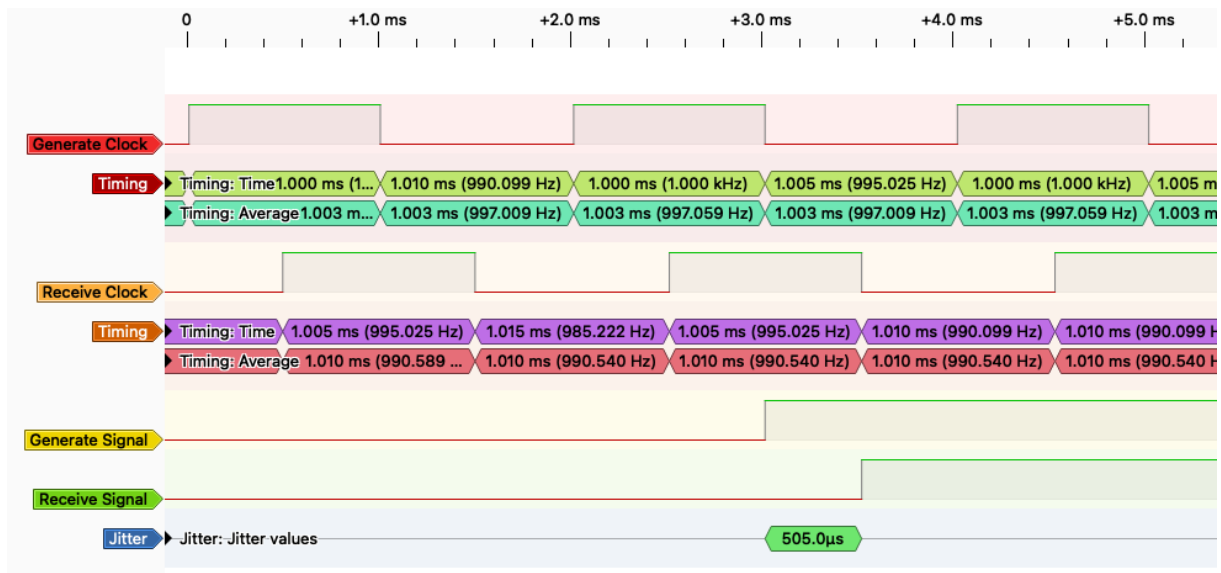


Figure 7.9: Test Harness PluseView Capture

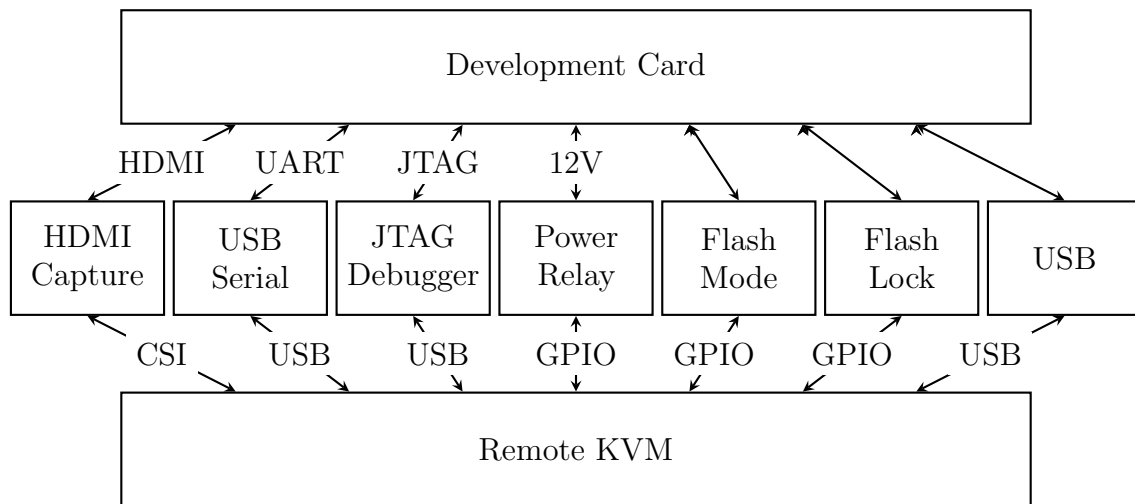
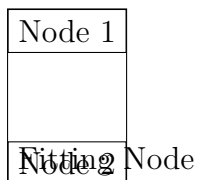


Figure 7.10: Development Hardware

Use the uBoot boot loader. This allows the development images to be loaded over TFTP, we have a dedicated tftp boot server.

Connected to a managed switch. The test harness is connected to this on a separate VLAN.



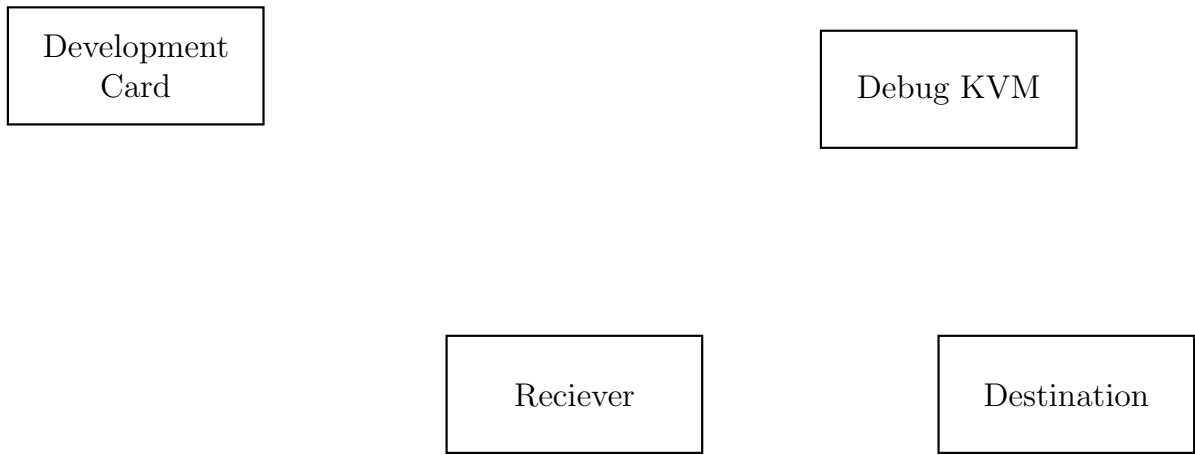
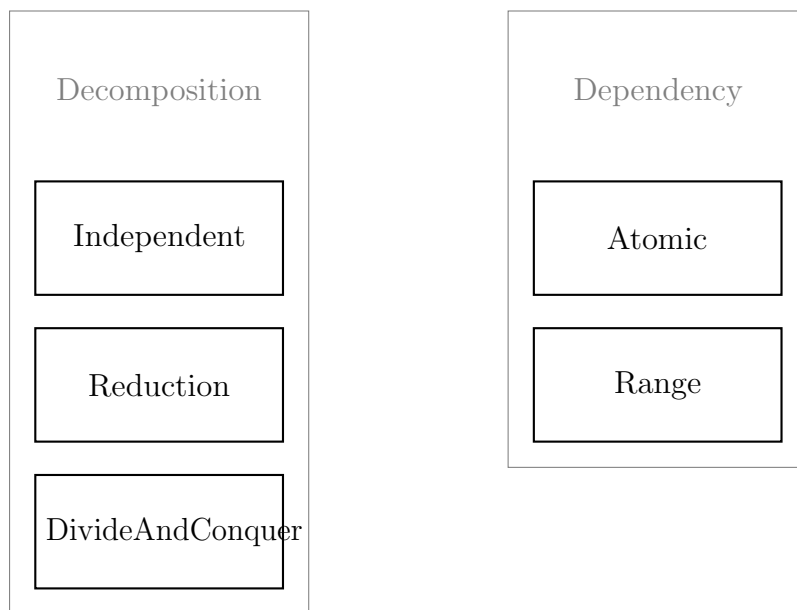


Figure 7.11: Heatmap for class distribution in dataset



Chapter 8

Testing

Chapter 9

Evaluation

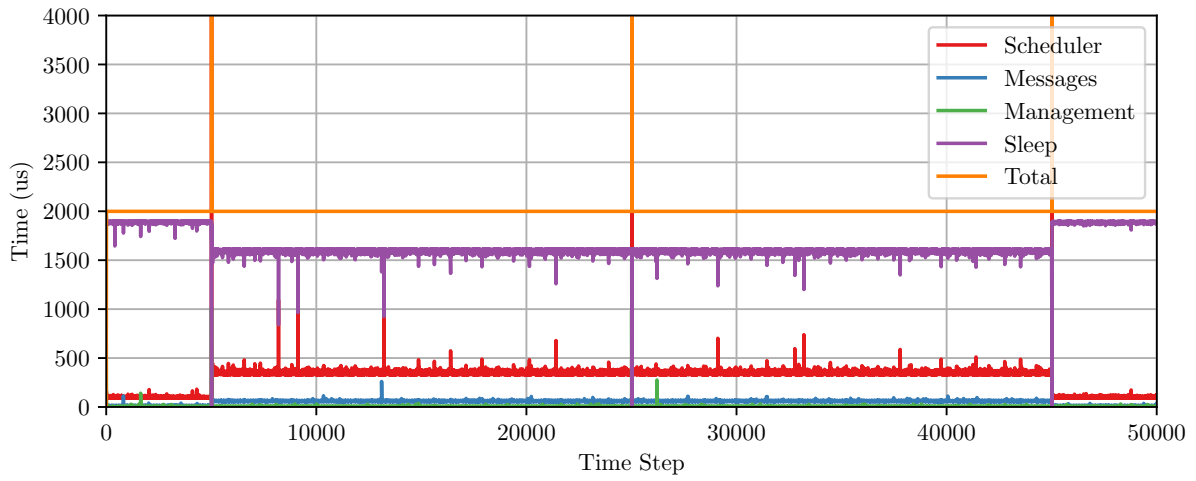


Figure 9.1: Elafry Demo Times

Chapter 10

Conclusion

References

- Abeni, Luca and Dario Faggioli (June 2020). “Using Xen and KVM as Real-Time Hypervisors”. In: *Journal of Systems Architecture* 106, p. 101709. ISSN: 13837621. DOI: 10.1016/j.sysarc.2020.101709. URL: <https://linkinghub.elsevier.com/retrieve/pii/S1383762120300035> (visited on 10/06/2023).
- Kavi, Krishna, Robert Akl, and Ali Hurson (Mar. 16, 2009). “Real-Time Systems: An Introduction and the State-of-the-Art”. In: *Wiley Encyclopedia of Computer Science and Engineering*. Ed. by Benjamin W. Wah. 1st ed. Wiley, pp. 2369–2377. ISBN: 978-0-471-38393-2 978-0-470-05011-8. DOI: 10.1002/9780470050118.ecse344. URL: <https://onlinelibrary.wiley.com/doi/10.1002/9780470050118.ecse344> (visited on 03/05/2024).
- Stallings, W. (2018). *Operating Systems: Internals and Design Principles, Global Edition*. Pearson Education. ISBN: 978-1-292-21430-6. URL: <https://books.google.co.uk/books?id=-76GEAAAQBAJ>.

Appendix A

Extra Information

Some more text ...